

Draft v0.1, not for distribution. Findings are preliminary, pending internal peer review and a response from the RAFA engineering team. Severities, counts, and wording may change before the final report.

DATSON360

Smart Contract Security Audit Report

DRAFT AUDIT REPORT

OCTOBER 2026

Target Protocol	RAFA Fund Protocol (Rafa-Protocol/rafa-fund)
Audited Commit	2ddd1c386e29a6d6d63eae080f48f0d417d54f61
Remediation Commit	Pending
Audit Review Period	September 28, 2026 to October 4, 2026
Auditor	DatSon360 Security Engagement Team
Report Status	Draft v0.1 (initial findings, no fixes reviewed yet)

1. Executive Summary

RAFA Fund Protocol is a non-custodial, tokenized fund system targeting Base and Ethereum. Investors deposit a single accounting asset (USDC on the planned deployments) and receive ERC-4626 vault shares. A RAFA-operated trader rebalances each fund, but only across assets, oracles, risk limits, and execution adapters approved in a chain-local protocol registry controlled by a RAFA Safe.

The system is built from four production contracts plus two venue adapters: `RafaFundV2` (vault, NAV, performance fee, guarded trading, in-kind exits), `RafaAssetRegistry` (protocol allowlist and per-asset risk policy), `FundFactoryV2` (registry of official funds), `ChainlinkPriceOracle`, and the `AerodromeAdapter` and `UniswapV3Adapter` execution paths. Contracts are immutable and do not use proxies. The earlier `BaseETF` and `FundFactory` prototypes reviewed by a prior firm have been removed from the repository and are out of scope.

This review covered commit `2ddd1c3`. DatSon360 identified **12 findings**: 0 Critical, 1 High, 2 Medium, 5 Low, and 4 Informational. No finding allows an unprivileged party to take funds directly. The High and Medium items concern what a compromised or careless privileged key can do, and how NAV-based share pricing behaves against oracle latency. All findings are open at the time of this draft.

The overall engineering quality is high. Access control, approval hygiene, oracle freshness, exposure caps, and the isolated in-kind exit path are all handled carefully and backed by 62 Hardhat tests, an adversarial adapter mock, and a clean Slither run. The remaining risk is concentrated in operational trust and economic design rather than in low-level Solidity defects.

Severity Category	Initial Findings	Resolved / Mitigated	Acknowledged / Pending
CRITICAL	0	0	0
HIGH	1	0	1
MEDIUM	2	0	2
LOW	5	0	5
INFORMATIONAL	4	0	4
Total	12	0	12

2. Audit Scope & Methodology

DatSon360 reviewed the Solidity contracts, deployment scripts, test suite, and architecture documentation in the primary repository. Manual line-by-line review was supplemented by a Slither pass, execution of the project's Hardhat suite, and hand-built scenarios exercising the trader, guardian, registry owner, and in-kind exit paths.

2.1 Repository & Commit Identification

- Repository: `https://github.com/Rafa-Protocol/rafa-fund`
- Audited Commit: `2ddd1c386e29a6d6d63eae080f48f0d417d54f61` (main, October 4, 2026)
- Compiler: Solidity 0.8.34, optimizer enabled, 1,000 runs
- Dependencies: OpenZeppelin Contracts 5.6.1, Hardhat 3.17.0

2.2 In-Scope Files

File Path	Logic & Component Description	Language / Version
<code>contracts/RafaFundV2.sol</code>	ERC-4626 vault. Share accounting, NAV with dual freshness windows, high-water-mark performance fee, guarded trade execution, exposure limits, pausing, and oracle-free in-kind redemption with reserved claims.	Solidity ^0.8.24
<code>contracts/RafaAssetRegistry.sol</code>	Owner-controlled allowlist. Per-asset oracle, adapter, decimals, price-age limits, protocol exposure cap, and admission/buy/sell switches.	Solidity ^0.8.24
<code>contracts/FundFactoryV2.sol</code>	Registry of official funds. Validates implementation ID, accounting asset, registry binding, and fee ceiling before listing a vault.	Solidity ^0.8.24
<code>contracts/oracles/ChainlinkPriceOracle.sol</code>	Derives an 18-decimal asset/accounting-asset price from two Chainlink USD feeds, with optional L2 sequencer uptime gating.	Solidity ^0.8.24
<code>contracts/adapters/AerodromeAdapter.sol</code>	Single-hop Aerodrome swaps between the accounting asset and an owner-configured route.	Solidity ^0.8.24
<code>contracts/adapters/UniswapV3Adapter.sol</code>	Single-pool Uniswap V3 <code>exactInputSingle</code> swaps against an owner-configured fee tier.	Solidity ^0.8.24
<code>contracts/interfaces/*.sol</code>	Registry, oracle, adapter, router, and aggregator interfaces.	Solidity ^0.8.24
<code>hardhat.config.ts, ignition/, scripts/</code>	Compiler settings, Ignition modules, Base Sepolia rehearsal script, and the contract-size check.	TypeScript

Mock contracts under `contracts/mocks/` were read for test context only and are excluded from findings. The off-chain web application and the asset discovery data under `assets/` are out of scope.

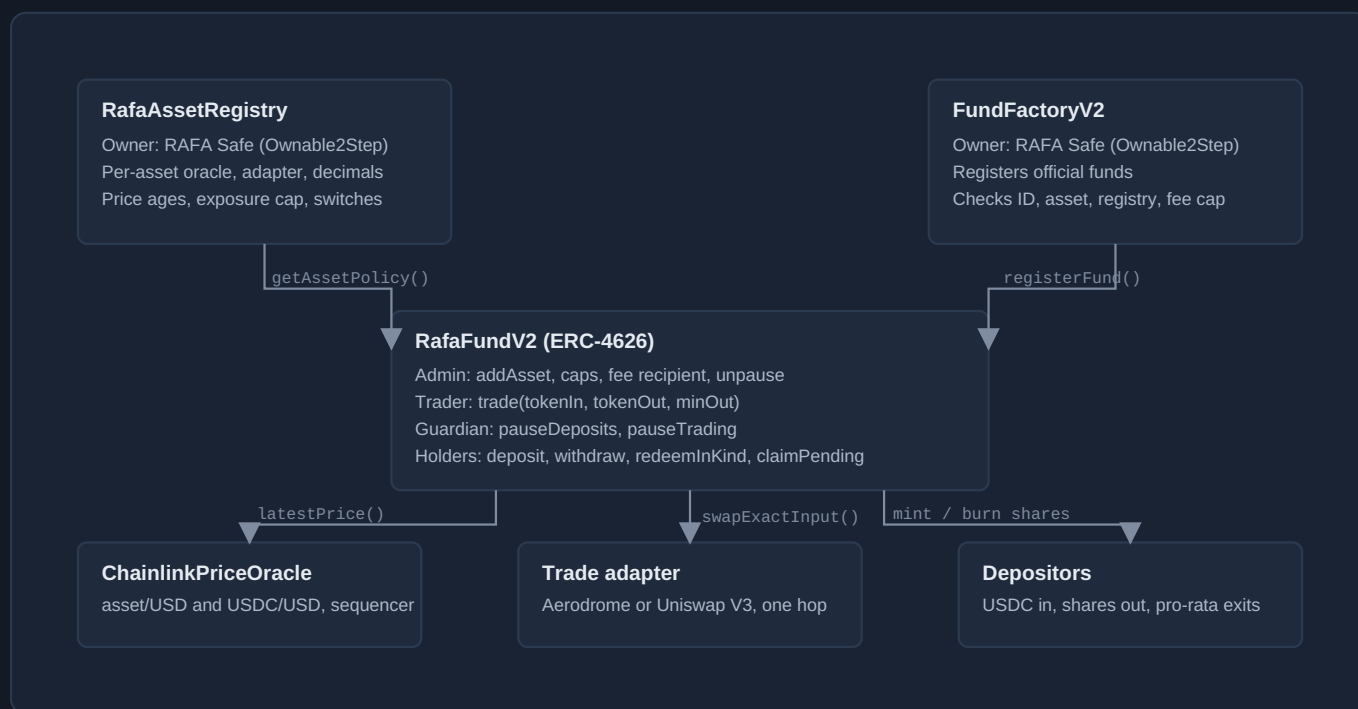
2.3 Audit Methodology

- **Architecture & design review.** Read `ARCHITECTURE.md` and `SECURITY_REVIEW.md`, mapped privileged roles and their reach, and built a threat model around the trader, guardian, fund admin, registry owner, and depositor.
- **Static & dynamic analysis.** Ran Slither against the production set, compiled under the pinned toolchain, and executed the 62-case Hardhat suite including the adversarial adapter and the 64-operation invariant sequence.
- **Manual code inspection.** Line-by-line review focused on share math and rounding, oracle freshness and manipulation, approval lifecycle, reentrancy across the self-call boundary, exposure enforcement, pause semantics, and fee accrual.
- **Scenario construction.** Hand-traced economic scenarios a static tool cannot see: slippage bleeding by a compromised trader, NAV arbitrage across oracle updates, registry hot-swaps, and dust grieving.

3. Protocol Architecture & System Design

Every fund holds the accounting asset plus up to ten registry-approved managed assets. Share price is NAV in accounting-asset units divided by share supply. Cash deposits and cash exits move only the accounting asset. The trader can only swap between the accounting asset and one managed asset per call, through the adapter the registry assigned to that asset, with a minimum output derived from the oracle price.

3.1 Core Architecture Diagram



3.2 Key System Roles

- **Registry Owner (RAFA Safe):** Configures and reconfigures every asset policy, including oracle and adapter addresses. Can disable admission, buys, or sells per asset at any time.
- **Factory Owner (RAFA Safe):** Lists and delists official funds. Has no power over a fund's assets.

- **Fund Admin (`DEFAULT_ADMIN_ROLE`)**: Adds and removes assets from the fund's subset, sets fund-level exposure limits, deposit cap, max trade value, fee recipient, and metadata. Only role that can unpause. Transfer is delayed via `AccessControlDefaultAdminRules`.
- **Trader (`TRADER_ROLE`)**: Executes swaps bounded by oracle slippage, per-trade value cap, and exposure limits.
- **Guardian (`GUARDIAN_ROLE`)**: Can pause deposits and trading. Cannot unpause.
- **Depositor**: Mints and burns shares. Always retains an oracle-independent, pause-independent in-kind exit.

3.3 Deposit, Trade, and Exit Workflows

Deposits accrue the performance fee first, then mint shares using OpenZeppelin's virtual-share math with a 12-decimal offset for a 6-decimal accounting asset. Cash withdrawals are limited to idle accounting-asset liquidity. Trades require the trader's `minAmountOut` to be at least the oracle-implied amount less `maxTradeSlippageBps`, verify exact input consumption and output receipt by balance delta, and re-check exposure after buys. In-kind redemption burns shares and delivers a proportional slice of every held token through a gas-bounded self-call; any delivery that fails becomes a reserved claim the receiver can later route to a different address.

4. Audit Assumptions & Operational Boundaries

4.1 External Dependency Assumptions

- **EVM compatibility**: Target chains follow standard EVM semantics; Base is an OP-stack L2 with a Chainlink sequencer uptime feed.
- **DEX router integrity**: The Aerodrome and Uniswap V3 routers behave per their public specifications. The vault does not trust adapter return values and verifies token movement itself.
- **Token behavior**: Managed assets are standard ERC-20s or Base B20 natives with reviewed ERC-20 behavior. Fee-on-transfer and rebasing tokens are out of design scope and would fail the exact-input check in `_executeTrade`.
- **Oracle availability**: Chainlink feeds honor their heartbeat and deviation thresholds. The registry owner configures `tradeMaxAge` and `valuationMaxAge` consistently with each feed's heartbeat.

4.2 Trust Assumptions

Role / Actor	Assumed Trust Level	Enforced On-Chain Boundaries
Registry Owner	Fully trusted	None against the owner itself. Oracle and adapter can be replaced for a live asset in one transaction (see MED-02).
Fund Admin	Semi-trusted	Cannot move tokens out except unsupported tokens via recovery. Fee rate and slippage ceiling are immutable per fund. Cannot exceed registry exposure caps.
Trader	Untrusted / Guarded	Can only swap accounting asset against approved holdings, output returns to vault, each trade bounded by oracle slippage and <code>maxTradeValueBps</code> . No cumulative limit (see HIGH-01).
Guardian	Semi-trusted	Pause only. A malicious guardian can halt deposits and trading but not exits.
Depositor	Untrusted	Subject to share math, deposit cap, and liquidity limits. Can always exit in kind.

5. Detailed Findings

All findings below are open. The status box under each item will be updated once the RAFA team responds and a remediation commit is reviewed.

5.1 High Severity Issues

HIGH

OPEN

HIGH-01: Compromised trader key can drain value through repeated max-slippage trades

contracts/RafaFundV2.sol: `trade()`, `_validateTrade()`, `MAX_TRADE_SLIPPAGE_BPS`

Description

Each trade is well bounded: the trader's `minAmountOut` must be at least the oracle-implied output minus `maxTradeSlippageBps`, and the trade's value may not exceed `maxTradeValueBps` of NAV. Both limits are per call. Nothing bounds how many trades the trader can execute in a block or a day, and nothing tracks cumulative realized loss against the oracle price.

`MAX_TRADE_SLIPPAGE_BPS` is 2,000 (20%) and `maxTradeValueBps` may be set to 10,000 (100% of NAV). A trader whose key is compromised, or who colludes with a searcher, can submit a buy and a sell back to back, each priced exactly at the slippage floor, and have a cooperating counterparty capture the spread on both legs. Repeating the round trip compounds the loss. At a 20% ceiling and full trade size, roughly 36% of NAV can leave the fund in a single round trip and the remainder follows in a handful of blocks.

Impact

The guardian can pause trading, but the response is reactive and the extraction can complete within a few blocks on an L2. The slippage ceiling is immutable per fund, so a fund launched with a generous value cannot be tightened later. Depositors have no on-chain protection beyond the configured parameters.

Recommendation

- Lower `MAX_TRADE_SLIPPAGE_BPS` to a level that reflects real execution needs on the configured pools (200 to 500 bps is typical for liquid pairs). Allow the admin to reduce, but never raise, a fund's slippage ceiling after deployment.
- Add a rolling-window limit on traded notional (for example, X% of NAV per 24 hours) so a single key compromise cannot cycle the full portfolio.
- Track cumulative oracle-relative loss per window and auto-pause trading when it crosses a threshold. This makes the guardian's pause a backstop rather than the only defense.
- Operationally, keep the trader key in a hardware or MPC signer with transaction policy limits.

Status: Open. Awaiting RAFA response. Severity assumes the trader key is a hot key, which the architecture documentation implies for agent-driven funds.

5.2 Medium Severity Issues

MEDIUM

OPEN

MED-01: NAV arbitrage across oracle latency with no entry or exit delay

contracts/RafaFundV2.sol: deposit(), redeem(), withdraw(), _calculateTotalAssets()

Description

Share price is computed from Chainlink prices at the moment of each deposit or cash exit. Chainlink feeds update on a heartbeat or a deviation threshold, so the on-chain price lags the market by up to the deviation band. A depositor who sees a managed asset rally off-chain can deposit before the feed updates, buying shares at the stale, lower NAV, and exit after the update. The same works in reverse for exits ahead of a drop. There is no deposit lock, no exit delay, and no entry or exit fee.

Impact

Value transfers from passive holders to whoever reacts fastest to off-chain price moves. On Base, with cheap gas and deviation thresholds of 0.5% to 1% on common feeds, this is a realistic, repeatable extraction. It scales with fund size and with the volatility of the managed assets.

Recommendation

- Introduce a minimum holding period or a short exit delay (hours, not days) so a position must bear at least one oracle update.
- Alternatively, charge a small symmetric entry/exit fee retained by the fund, sized to exceed the feed's deviation threshold.
- Document the chosen mitigation and the feed parameters it was sized against.

Status: Open. Severity may be revised down if RAFA restricts deposits to allowlisted or KYC'd wallets on the production launch.

MEDIUM**OPEN**

MED-02: Registry owner can hot-swap oracle and adapter for live assets with no timelock

```
contracts/RafaAssetRegistry.sol: configureAsset(); contracts/RafaFundV2.sol:  
_assetPolicy()
```

Description

Every fund reads its asset policy from the registry on each call. `configureAsset` can be called again for an already-configured asset and replaces its oracle, adapter, decimals, price ages, and exposure cap in a single owner transaction. The registry uses `Ownable2Step` but no delay. Funds and their depositors have no window to react.

A malicious or compromised registry owner could point an asset to an oracle that reports an inflated price, which causes `_accruePerformanceFee` to mint fee shares against phantom profit; the fee recipient can then exit in kind for real assets. Pointing to a deflated price lets a colluding depositor mint cheap shares. Pointing to a malicious adapter would still be caught by the vault's exact-input and min-output checks, so the oracle path is the dangerous one.

Impact

Full value extraction is possible by the registry owner. This is a trust assumption the protocol documents, but the trust is currently absolute and instant rather than delayed and observable.

Recommendation

- Route `configureAsset` reconfigurations (not first-time configuration or the status switches) through a timelock of at least 24 to 48 hours.
- Consider making `decimals` and `oracle` immutable after first configuration unless the asset is first disabled for buys and admission.
- Emit a distinct event when an existing policy is overwritten so monitors can alert on it.

Status: Open. Awaiting confirmation of the Safe's signer threshold and whether a timelock module is planned.

5.3 Low Severity Issues

LOW

OPEN

LOW-01: Settlement price freshness is enforced as a side effect of fee accrual

contracts/RafaFundV2.sol: `_accruePerformanceFee()`, `withdraw()`, `redeem()`

Description

The README states that deposits, cash exits, fees, and trades use the strict `tradeMaxAge` window. In the code, `withdraw` and `redeem` price shares through `totalAssets()`, which uses the looser `valuationMaxAge`. The strict window is only enforced because `_accruePerformanceFee` calls `settlementTotalAssets()` first and reverts on a stale price. The guarantee is correct today but depends on the ordering of an unrelated call. A future refactor that short-circuits fee accrual when `performanceFeeBps == 0` would silently weaken the exit path.

Recommendation

Add an explicit freshness check in the cash entry and exit paths, or override `_convertToShares` and `_convertToAssets` to take the settlement NAV when called from state-changing functions. Add a test that asserts `withdraw` reverts on a price older than `tradeMaxAge` on a fund with a zero performance fee.

Status: Open.

LOW

OPEN

LOW-02: removeAsset can be blocked permanently by a dust donation

contracts/RafaFundV2.sol: `removeAsset()`, `recoverUnsupportedToken()`

Description

`removeAsset` reverts unless the raw `balanceOf` is zero. Anyone can send one wei of the token to the vault. The admin cannot recover it because the asset is supported, and the trader cannot sell it because a one-wei swap will fail at the DEX. The asset stays in `_activeAssets`, continues to count toward `MAX_ASSETS`, and keeps an oracle call in every NAV computation.

Recommendation

Allow removal when the balance is below a small admin-set dust threshold, and sweep the dust to the fee recipient or leave it stranded. Alternatively, permit `recoverUnsupportedToken` for a supported asset when the registry has disabled both buys and admission for it.

Status: Open.

LOW**OPEN****LOW-03: In-kind exits skip performance fee during an oracle outage**

```
contracts/RafaFundV2.sol: redeemInKind(), _tryAccruePerformanceFee()
```

Description

`redeemInKind` wraps fee accrual in a try/catch so exits remain available when a price is stale. If NAV is above the high-water mark at that moment, the exiting holder leaves at the gross price and the fee on their slice is never collected. The fee recipient bears the loss. Remaining holders are unaffected because the high-water mark is unchanged.

Recommendation

Accept the leakage as the cost of a liveness guarantee and document it, or track an "unaccrued fee" estimate at the last known NAV and settle it on the next successful accrual. The first option is simpler and likely correct for the stated design goals.

Status: Open. Likely to be acknowledged rather than fixed.

LOW**OPEN****LOW-04: Registry does not verify configured decimals against the token**

```
contracts/RafaAssetRegistry.sol: configureAsset()
```

Description

`decimals` is supplied by the owner and never compared to the token's own `decimals()`. The design note explains this is to support B20 natives with empty bytecode. For ordinary tokens, a typo (6 instead of 18) would misvalue NAV by twelve orders of magnitude, which would either brick deposits through the exposure check or let the first depositor mint shares against wildly inflated NAV.

Recommendation

When `assetToken.code.length != 0`, call `decimals()` in a try/catch and revert on mismatch. Keep the unchecked path only for tokens with no bytecode.

Status: Open.

LOW**OPEN****LOW-05: Factory validates fee ceiling but not slippage ceiling or admin transfer delay**`contracts/FundFactoryV2.sol: registerFund()`**Description**

An "official" fund must respect `maximumPerformanceFeeBps`, but the factory does not check `maxTradeSlippageBps` or the `AccessControlDefaultAdminRules` delay. A fund with the maximum 20% slippage and a zero admin transfer delay can be listed. Users rely on the official list as a quality signal.

Recommendation

Add immutable factory maximums for slippage and a minimum admin transfer delay, and check both in `registerFund`.

Status: Open.

5.4 Informational**INFO****OPEN****INFO-01: Floating pragma in production contracts**`All contracts: pragma solidity ^0.8.24; compiled with 0.8.34`

Lock the pragma to the compiler used for deployment so verified source and bytecode match exactly. The prior audit of the `BaseETF` prototype raised the same point.

Status: Open.

INFO**OPEN****INFO-02: Chainlink answer not checked against aggregator bounds; answeredInRound is deprecated**`contracts/oracles/ChainlinkPriceOracle.sol: _readFeed()`

The oracle rejects non-positive answers, zero timestamps, and future timestamps, and reads `min(updatedAt)` across both feeds, which is correct. Two notes. First, Chainlink aggregators clamp to `minAnswer` and `maxAnswer` during extreme moves, so a feed can report a wrong but "fresh" price. Consider reading those bounds and rejecting answers at the edge. Second, `answeredInRound` is deprecated on OCR2 feeds and the check adds no safety there; it can be dropped.

Status: Open.

INFO

OPEN

INFO-03: Deposit cap applies to NAV, and exposure breaches surface as a cap error

contracts/RafaFundV2.sol: `maxDeposit()`, `_depositWithMinimum()`

`depositCap` is compared against current NAV, so price appreciation alone can close a fund to new deposits. When exposures are out of limits, `maxDeposit` returns zero and the user sees `DepositCapExceeded(assets, 0)`, which is misleading.

Consider a distinct `ExposureLimitsBreached` error and documenting that the cap is NAV-based.

Status: Open.

INFO

OPEN

INFO-04: Naming and documentation drift

contracts/FundFactoryV2.sol: `FundCreated`; `README.md`

`FundFactoryV2` does not deploy anything; it registers vaults deployed elsewhere, yet emits `FundCreated`. Indexers may assume a creation transaction. The README's claim that cash exits use the strict window is true only via the coupling described in LOW-01. Both are worth tightening before the final report.

Status: Open.

6. Architectural Considerations & Code Quality

Beyond the findings above, DatSon360 reviewed the codebase for durability and alignment with current Solidity practice. The following were checked and found sound.

6.1 What is done well

- **Inflation attack.** The 12-decimal virtual share offset for USDC makes the classic first-depositor attack uneconomic. Managed-asset donations are also absorbed by the offset.
- **Approval lifecycle.** `forceApprove(adapter, amountIn)` before the swap and `forceApprove(adapter, 0)` after. Adapters mirror the pattern against the router. No standing allowances.
- **Adapter distrust.** The vault measures input consumed and output received by balance delta and ignores the adapter's return value. The `AdversarialTradeAdapter` test confirms this.
- **Reentrancy.** All state-changing entry points are `nonReentrant`. The self-call boundary in `redeemInKind` correctly leaves `deliverInKindAsset` unguarded and restricts it to `address(this)`. A token with transfer hooks cannot reenter `claimPending` or any exit path while the lock is held.
- **Reserved claims.** `_availableBalance` subtracts `totalPendingClaims` everywhere NAV, liquidity, or exposure is computed, so a stuck RWA transfer cannot be counted or spent twice.
- **Pause asymmetry.** Guardian can pause, only admin can unpause, exits are never pausable.
- **Fee math.** Dilution-based fee minting and high-water-mark updates are correct, including the reset to `1e18` when supply reaches zero.
- **Sequencer handling.** Grace-period logic is correct and `startedAt == 0` is treated as unsafe.

6.2 Test Coverage

Area	Observation	Suggested additions
Vault core	62 cases, 88% line coverage per the project's own review, 64-operation invariant sequence.	Stale price on a zero-fee fund (LOW-01); multi-trade loss bound (HIGH-01); dust-blocked removal (LOW-02).
Oracle	Sequencer, negative answer, and future timestamp paths covered.	Clamped answer at <code>minAnswer</code> / <code>maxAnswer</code> (INFO-02).
Registry	Status switches and first-configuration defaults covered.	Reconfiguration of a live asset and its effect on an existing fund's NAV (MED-02).
Adapters	Mock routers exercise happy path and revert path.	Fork tests against real Aerodrome and Uniswap V3 pools on Base before mainnet.

6.3 Gas and Maintainability Notes

- `_assetPolicy` is read from the registry on every loop iteration in `_calculateTotalAssets` and `_exposuresWithinLimits`. With up to ten assets this is two external calls per asset per NAV read. Acceptable on L2, but worth caching within a single call if the asset ceiling ever rises.
- `_checkGuardianOrAdmin` reaches the revert through a second `_checkRole` call. A direct custom error would be clearer.
- Custom errors, `++i` increments, and immutables are used throughout. No string reverts remain.

7. Appendices

Appendix A: File Scope Verification

SHA-256 hashes of the production contracts at commit `2ddd1c386e29a6d6d63eae080f48f0d417d54f61`.

File Path	Lines	SHA-256 Hash
contracts/RafaFundV2.sol	794	fd0f6c972854c2894556f907b7d4a024841fe071bf22aaba21ad40cac4d5ea0e
contracts/ RafaAssetRegistry.sol	124	8c80310bb7f81f755c3225dcadb8e0f8d6fd5eab959ad3246d53d580aed89b4a
contracts/ FundFactoryV2.sol	115	a7912e9ea3e80f954d39f591d303483dd597c8202948b507c770250e725420ea
contracts/oracles/ ChainlinkPriceOracle.sol	92	0db5b993dabe06ac59bdbc2de21259e7cc8d315645d69ee9f6ffade9b8202c42
contracts/adapters/ AerodromeAdapter.sol	94	3c75780ff70e74f84b905ae17c3b535d74fad1910b3a655ca38a383572733eba
contracts/adapters/ UniswapV3Adapter.sol	91	4bb2c26a0d2e69fd53cd24fac8e368a10d04ec2e5afd53622b79358c0cc974e1

Appendix B: Severity Classification

Severity	Definition used in this report
CRITICAL	Unprivileged loss of funds or permanent freezing of user assets.
HIGH	Loss of funds requiring a privileged key or specific but plausible conditions.
MEDIUM	Bounded value leakage, or loss of funds requiring a fully trusted party to act maliciously without a delay.
LOW	Griefing, fragile invariants, or misconfiguration risk with limited impact.
INFORMATIONAL	Code quality, documentation, or defense-in-depth suggestions.

Appendix C: Auditor Disclosures & Disclaimer

DatSon360 is an independent smart contract security firm. No team member involved in this engagement holds a financial stake, token allocation, or advisory role in RAFA Protocol or its related entities. Fees for this engagement were fixed in advance and do not depend on the findings.

A security audit is a time-bounded review of a specific commit. It is not a guarantee that the code is free of defects, and it does not replace continuous monitoring, a bug bounty, fork testing against live venues, and sound operational key management. This draft has not completed internal peer review and should not be relied on or published.

DatSon360 Lead Auditor

Signed: pending final review

Hash: to be issued with final report

RAFA Protocol Lead Engineer

Acknowledged: pending

Date: pending